

每日三句精選 | Loop Engineering 影片摘要

1. 什麼是 Loop Engineering?

原句

"Don't prompt your agent — design the loop that prompts your agent."

— Peter Steinberger (OpenClaw創辦人)

唔好再寫 prompt 俾 agent，而係設計一個 loop 去 prompt 你的 agent。

寫法分析

- 核心轉變：從 Human in the Loop → Agent in the Loop
- 運作方式：人類設計目標、工具、驗收標準 → AI 自行迭代執行、檢查、修正 → 完成或停下來回報
- 關鍵人物：Peter Steinberger, Boris Cherny (Claude Code), Addy Osmani (Google) 同期提出類似概念

優化版本

"以前：你寫 prompt → AI 回你 → 你睇 → 你改 → 再 prompt 而家：你設計 loop → AI 自己跑測試、讀錯誤、再改 → 直到完成"

(審核角色由人類部分轉移到 AI，人類只需定義規則)

2. Loop 的兩個核心機制

原句

"The loop is formed by two questions: when to start, and when to stop."

Loop 的形成取決於兩個問題：什麼時候開始，什麼時候停止。

寫法分析

- Trigger (觸發器)：決定 loop 何時開始。可以是事件 (如 GitHub PR 開啟)、排程 (每天定時)、或手動觸發
- Verifiable Goal (可驗證目標)：決定 loop 何時停止。需要定義清楚「什麼叫完成」以及「AI 如何檢查自己完成了」

商業應用

實例：叫 AI 做十張 YouTube 封面圖 → 用四條標準打分 → 把沒過關的挑出來重做 → 跑兩輪後交最高的三張

3. Loop Engineering 與其他名詞的關係

原句

"Prompt Engineering → Context Engineering → Harness Engineering → Loop Engineering"

這是 AI 協作技術的演進路徑。

寫法分析

- Prompt Engineering：強調跟 AI 溝通的技巧，把話說清楚 (語氣、格式、限制)
- Context Engineering：在對的時機給 AI 正確且不多不少的資訊
- Harness Engineering：設定規矩和邊界——給 AI 辦公室、工具、流程、權限
- Loop Engineering：把 Human in the Loop 變成 Agent in the Loop，人類將 feedback 邏輯標準化

變體練習

- 關鍵差異：前三個 focus 在「輸入」，Loop Engineering focus 在「迭代過程的自動化」

4. 如何設定驗收標準

原句

"Turning abstract concepts into clear standards is where most people get stuck."

把抽象概念變成清楚的標準是大多數人實作時的卡點。

寫法分析

- 方法一：Rubric 評分：選定幾個面向（如風格人設、語法用詞、內容主題），每個面向定義 1-5 分的分級標準
- 方法二：二元檢查清單：把品質拆成一串 Yes/No 問題（如：開頭有冇重點？有冇冗詞？專有名詞有冇用錯？）
- 編程任務：所有測試通過、TypeScript 沒報錯、lint 沒違規、build 成功——機器可以直接檢查

商業應用

Rubric 比二元檢查更靈活，但二元檢查更穩定、更容易判斷。建議：先從檢查清單開始。

5. 什麼任務值得做 Loop?

原句

"Don't blindly follow trends. Ask: does my workflow actually need this?"

不要盲目跟風。先問自己的工作流是否真的需要。

寫法分析

- 條件一：重複發生：一次性任務直接寫 prompt 就好，沒必要設計 loop
- 條件二：標準清楚：要能講出「什麼叫這件事做完了」——可以是 CI 全綠、網站載入低於 2 秒
- 條件三：扛得住成本：Loop 不是免費的。手動 prompt 3 次就解決的事，loop 跑了 20 輪還不停——帳單會很有教育意義

變體練習

- 例外情況：快速做 demo 時可以用 loop 先跑主功能，細節再由人接手調整

6. Loop 的三個常見問題

原句

"Without a hard stop, your loop becomes a token black hole."

沒有硬性停止條件，loop 就會變成 token 黑洞。

寫法分析

- 問題一：不知道何時該停：解決：一定要有 hard stop——最多跑幾輪、最多跑多久、最多花多少 token、三輪沒進展就停
- 問題二：碰了不該碰的東西：解決：定義邊界——不能刪測試、不能改公開 API、不能重構不該碰的架構
- 問題三：偏離太遠：讓 agent 自己迭代一整晚，一個不小心可能偏到十萬八千里。需要退路計劃

商業應用

大多數人其實不需要 AI 不眠不休跑 27 小時。Human in the Loop 往往更準、更有效率、也更便宜。

今日學習重點

概念	重點	應用	注意
Trigger	事件/排程/手動啟動	GitHub PR, 每日任務	要明確定義
Verifiable Goal	完成條件+檢查方式	測試通過, Rubric 評分	抽象任務最難
Hard Stop	輪數/時間/Token 上限	3 輪沒進展就停	Loop 不失控關鍵
邊界定義	不能改的範圍	不能刪測試/改 API	防止偏離
成本控制	Token 預算	手動 vs Loop 取捨	一般人沒有無限預算

老師提示：Loop Engineering 真正的難點不是技術，而是你能不能把抽象的需求轉化成清楚的標準。先從一個小 loop 開始——選一個重複發生、標準夠清楚的任務，設好 hard stop，觀察幾輪，再逐步擴大。記住：普通人學 Loop Engineering 的第一步不是馬上搞一個，而是判斷哪些需求可以用上。